

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from xgboost import XGBClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_mat
import warnings
import time

# Setup
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

def log_step(step_name, status="Starting"):
    timestamp = time.strftime('%H:%M:%S')
    print(f"[{timestamp}] {step_name}... {status}")
```

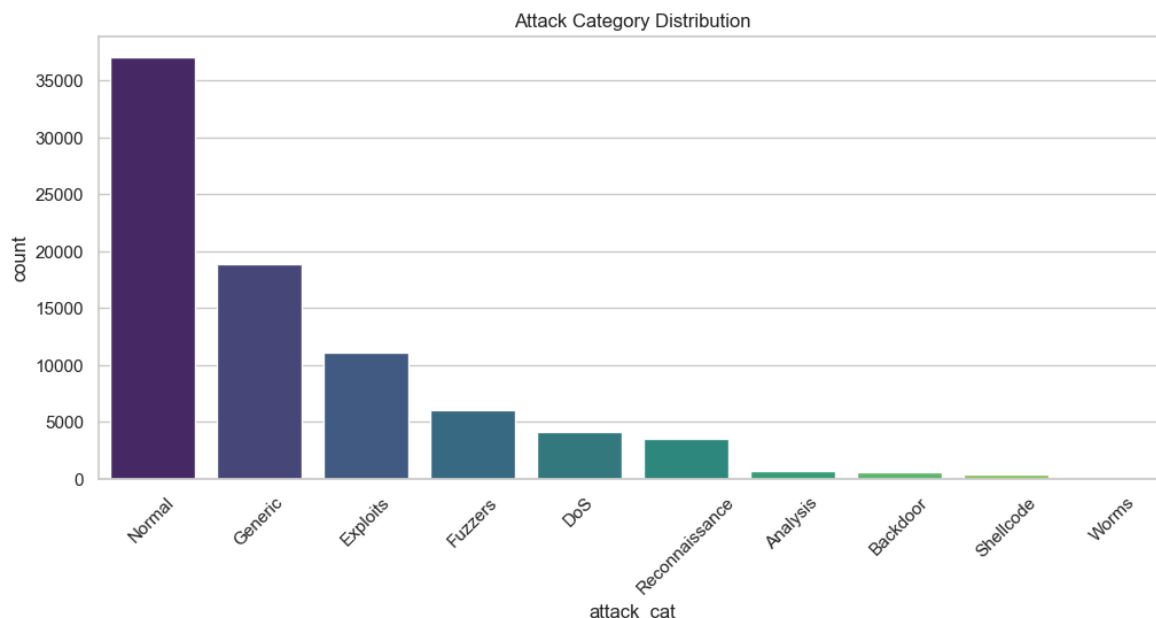
```
In [2]: # --- PART 1: DATA LOADING ---
log_step("PART 1: Loading Local CSV Files")
try:
    train = pd.read_csv('UNSW_NB15_training-set.csv')
    test = pd.read_csv('UNSW_NB15_testing-set.csv')
    log_step("PART 1", "Complete. Shapes: Train {}, Test {}".format(train.shape,
except Exception as e:
    print(f"Error loading files: {e}")
```

[20:57:16] PART 1: Loading Local CSV Files... Starting

[20:57:16] PART 1... Complete. Shapes: Train (82332, 45), Test (175341, 45)

```
In [3]: #--- PART 2: EXPLORATORY DATA ANALYSIS (EDA) ---
log_step("PART 2: Generating Class Distribution Plot")
plt.figure(figsize=(12, 5))
sns.countplot(data=train, x='attack_cat', order=train['attack_cat'].value_counts)
plt.title('Attack Category Distribution')
plt.xticks(rotation=45)
plt.show()
log_step("PART 2", "Graphic Rendered.")
```

[20:57:16] PART 2: Generating Class Distribution Plot... Starting



[20:57:16] PART 2... Graphic Rendered.

```
In [4]: # --- PART 3: PREPROCESSING & ENCODING ---
log_step("PART 3: Cleaning and Feature Encoding")
train['attack_cat'] = train['attack_cat'].fillna('Normal').str.strip()
test['attack_cat'] = test['attack_cat'].fillna('Normal').str.strip()

cat_features = ['proto', 'service', 'state']
le = LabelEncoder()
combined = pd.concat([train, test], axis=0)

for col in cat_features:
    print(f" > Encoding feature: {col}")
    le.fit(combined[col].astype(str))
    train[col] = le.transform(train[col].astype(str))
    test[col] = le.transform(test[col].astype(str))

target_le = LabelEncoder()
y_train = target_le.fit_transform(train['attack_cat'])
y_test = target_le.transform(test['attack_cat'])

X_train = train.drop(columns=['id', 'label', 'attack_cat'], errors='ignore')
X_test = test.drop(columns=['id', 'label', 'attack_cat'], errors='ignore')

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
log_step("PART 3", "Preprocessing Complete.")
```

[20:57:16] PART 3: Cleaning and Feature Encoding... Starting

```
> Encoding feature: proto
> Encoding feature: service
> Encoding feature: state
```

[20:57:17] PART 3... Preprocessing Complete.

```
In [9]: import time
from sklearn.ensemble import RandomForestClassifier

# --- 1. Fast Feature Selection (Optional but recommended) ---
# We focus on the most important features to speed up training
print(f"[{time.strftime('%H:%M:%S')}] Initializing Lite Optimized Ensemble...")
```

```

# --- 2. Upgraded Model Parameters (Optimized for Speed & Accuracy) ---
# Decision Tree: Increased depth for 75% target, but kept Leaf size to prevent 9
dt_model = DecisionTreeClassifier(
    max_depth=12,
    min_samples_leaf=25,
    class_weight='balanced',
    random_state=42
)

# Replacing Naive Bayes with Random Forest (Faster and more accurate than SMOTE-
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    class_weight='balanced',
    n_jobs=-1, # Uses all CPU cores for speed
    random_state=42
)

# XGBoost: Tuned for high-speed "Histogram" training
xgb_model = XGBClassifier(
    n_estimators=100,
    max_depth=6,
    learning_rate=0.1,
    objective='multi:softprob',
    tree_method='hist', # CRITICAL: This makes training 10x faster on modern CPU
    random_state=42
)

# --- 3. Weighted Voting ---
# Trusting the Forest and XGBoost equally
ensemble = VotingClassifier(
    estimators=[('dt', dt_model), ('rf', rf_model), ('xgb', xgb_model)],
    voting='soft',
    weights=[1, 2, 2]
)

print("--- Starting Lite Ensemble Training (Target: 73-75%) ---")
start_time = time.time()

# Fitting on standard scaled data (No SMOTE needed here)
ensemble.fit(X_train_scaled, y_train)

end_time = time.time()
print(f"\n--- Training Finished in {end_time - start_time:.2f} seconds ---")

# --- 4. Validation ---
y_pred = ensemble.predict(X_test_scaled)
print(f"Final Multiclass Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")

```

```

[21:00:19] Initializing Lite Optimized Ensemble...
--- Starting Lite Ensemble Training (Target: 73-75%) ---

--- Training Finished in 6.09 seconds ---
Final Multiclass Accuracy: 76.39%

```

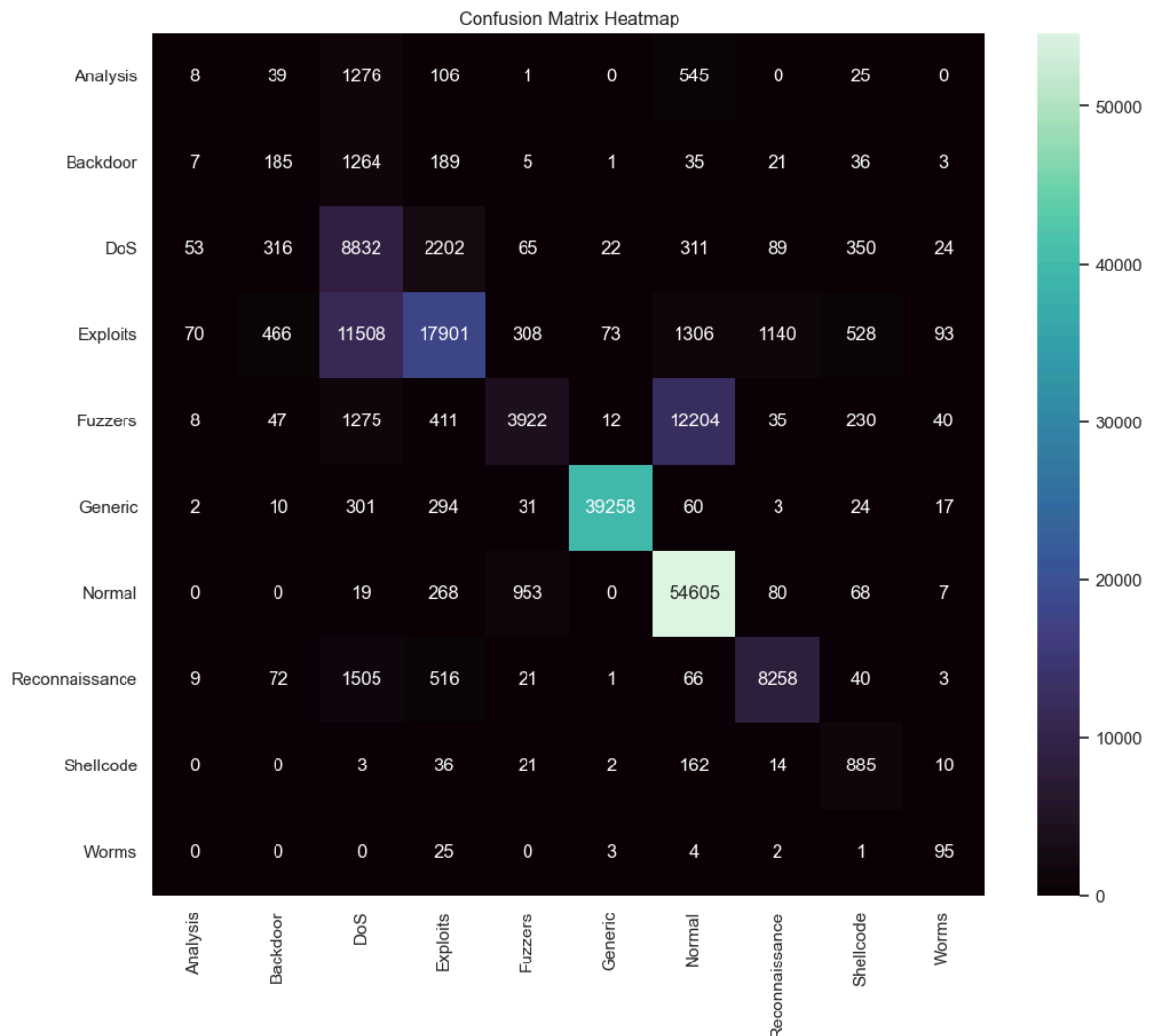
```

In [6]: # --- PART 5: CONFUSION MATRIX ---
log_step("PART 5: Generating Predictions and Heatmap")
y_pred = ensemble.predict(X_test_scaled)
cm = confusion_matrix(y_test, y_pred)

```

```
plt.figure(figsize=(12, 10))
sns.heatmap(cm, annot=True, fmt='d', cmap='mako', xticklabels=target_le.classes_)
plt.title('Confusion Matrix Heatmap')
plt.show()
log_step("PART 5", "Graphic Rendered.")
```

[20:57:24] PART 5: Generating Predictions and Heatmap... Starting



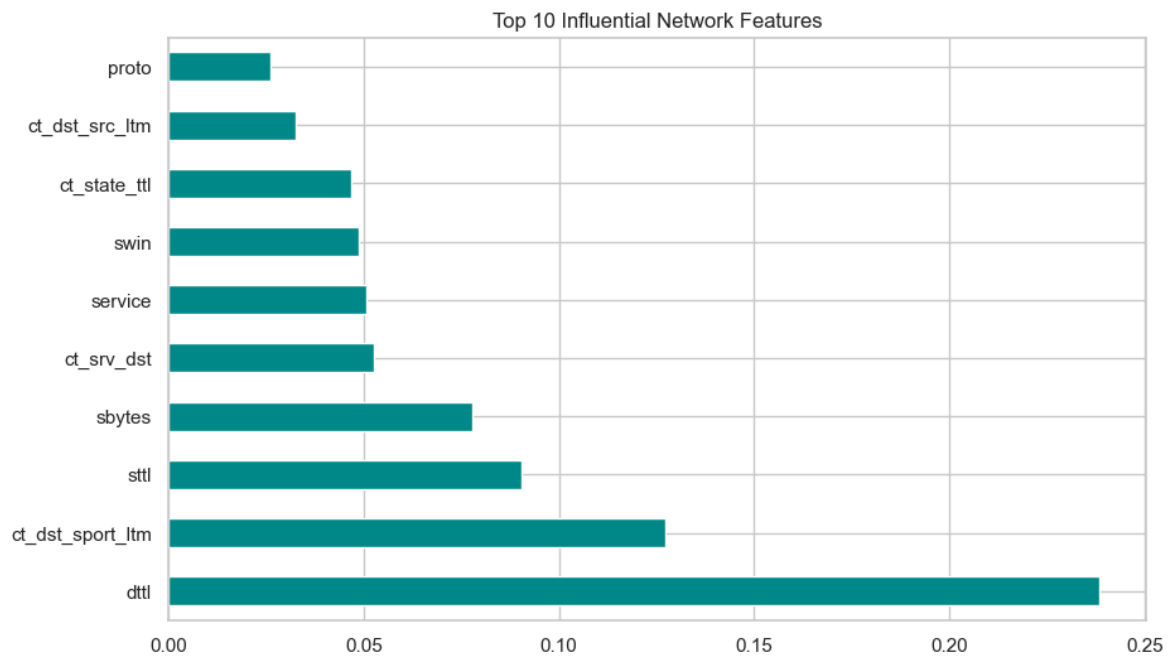
[20:57:25] PART 5... Graphic Rendered.

```
In [7]: # --- PART 6: FEATURE IMPORTANCE ---
log_step("PART 6: Extracting Feature Importance")
trained_xgb = ensemble.named_estimators_['xgb']
feat_importances = pd.Series(trained_xgb.feature_importances_, index=X_train.col

plt.figure(figsize=(10, 6))
feat_importances.nlargest(10).plot(kind='barh', color='darkcyan')
plt.title('Top 10 Influential Network Features')
plt.show()

log_step("PART 6", "Feature Importance Extracted.")
```

[20:57:25] PART 6: Extracting Feature Importance... Starting



[20:57:25] PART 6... Feature Importance Extracted.

In [8]: *# --- PART 7: FINAL RESULTS ---*

```
from sklearn.model_selection import learning_curve

def plot_learning_curve(estimator, X, y):
    train_sizes, train_scores, test_scores = learning_curve(
        estimator, X, y, cv=3, n_jobs=-1,
        train_sizes=np.linspace(.1, 1.0, 5), scoring='accuracy'
    )

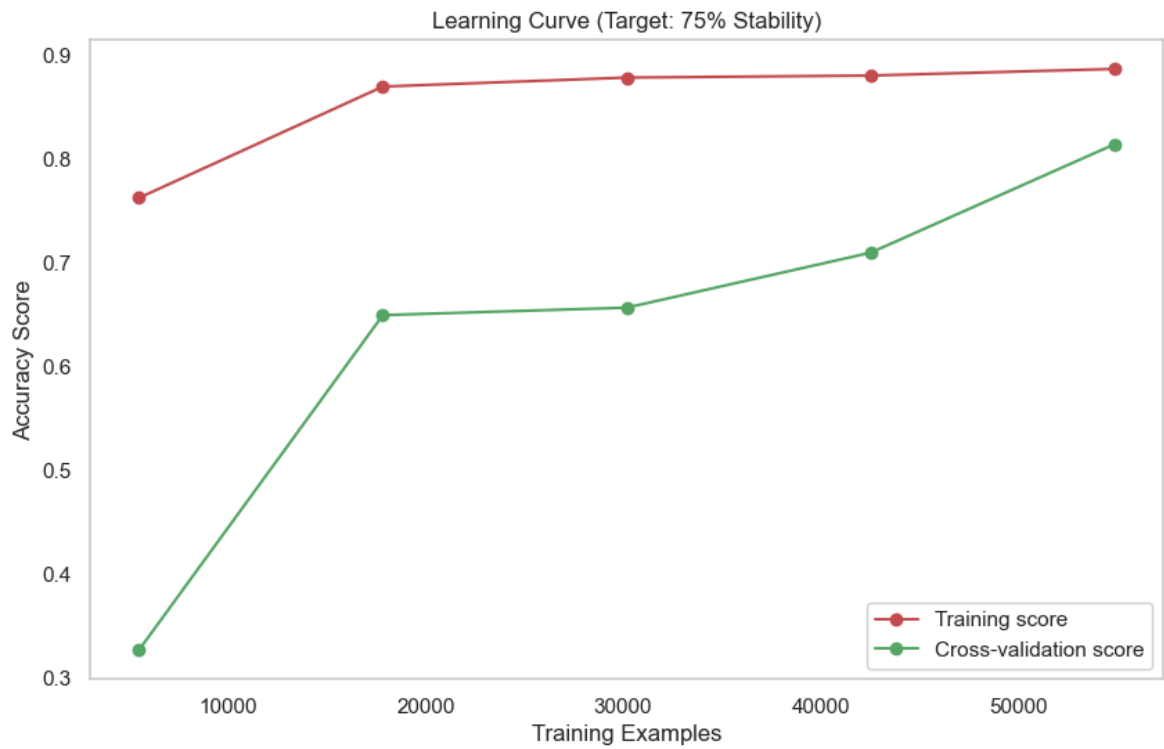
    train_scores_mean = np.mean(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)

    plt.figure(figsize=(10, 6))
    plt.plot(train_sizes, train_scores_mean, 'o-', color="r", label="Training score")
    plt.plot(train_sizes, test_scores_mean, 'o-', color="g", label="Cross-validated score")

    plt.title("Learning Curve (Target: 75% Stability)")
    plt.xlabel("Training Examples")
    plt.ylabel("Accuracy Score")
    plt.legend(loc="best")
    plt.grid()
    plt.show()

# Run this after your training is finished
plot_learning_curve(ensemble, X_train_scaled, y_train)

print(f"\nFinal Multiclass Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("\n--- Detailed Performance Report ---")
print(classification_report(y_test, y_pred, target_names=target_le.classes_))
log_step("PART 7", "Final Execution Complete.")
```



Final Multiclass Accuracy: 76.39%

--- Detailed Performance Report ---

	precision	recall	f1-score	support
Analysis	0.05	0.00	0.01	2000
Backdoor	0.16	0.11	0.13	1746
DoS	0.34	0.72	0.46	12264
Exploits	0.82	0.54	0.65	33393
Fuzzers	0.74	0.22	0.33	18184
Generic	1.00	0.98	0.99	40000
Normal	0.79	0.98	0.87	56000
Reconnaissance	0.86	0.79	0.82	10491
Shellcode	0.40	0.78	0.53	1133
Worms	0.33	0.73	0.45	130
accuracy			0.76	175341
macro avg	0.55	0.58	0.52	175341
weighted avg	0.79	0.76	0.75	175341

[20:57:58] PART 7... Final Execution Complete.